

**EXERCISE 12.14**

Hard: Implement `map` in terms of `traverse` as a method on `Traverse[F]`. This establishes that `Traverse` is an extension of `Functor` and that the `traverse` function is a generalization of `map` (for this reason we sometimes call these *traversable functors*). Note that in implementing `map`, you can call `traverse` with *your choice* of `Applicative[G]`.

```
trait Traverse[F[_]] extends Functor[F] {

  def traverse[G[_],A,B](fa: F[A])(f: A => G[B]) (
    implicit G: Applicative[G]): G[F[B]] =
    sequence(map(fa)(f))

  def sequence[G[_],A](fga: F[G[A]])(
    implicit G: Applicative[G]): G[F[A]] =
    traverse(fga)(ga => ga)

  def map[A,B](fa: F[A])(f: A => B): F[B] = ???
}
```

But what is the relationship between `Traverse` and `Foldable`? The answer involves a connection between `Applicative` and `Monoid`.

12.7.1 From monoids to applicative functors

We’ve just learned that `traverse` is more general than `map`. Next we’ll learn that `traverse` can also express `foldMap` and by extension `foldLeft` and `foldRight`! Take another look at the signature of `traverse`:

```
def traverse[G[_]:Applicative,A,B](fa: F[A])(f: A => G[B]): G[F[B]]
```

Suppose that our `G` were a type constructor `ConstInt` that takes any type to `Int`, so that `ConstInt[A]` throws away its type argument `A` and just gives us `Int`:

```
type ConstInt[A] = Int
```

Then in the type signature for `traverse`, if we instantiate `G` to be `ConstInt`, it becomes

```
def traverse[A,B](fa: F[A])(f: A => Int): Int
```

This looks a lot like `foldMap` from `Foldable`. Indeed, if `F` is something like `List`, then what we need to implement this signature is a way of combining the `Int` values returned by `f` for each element of the list, and a “starting” value for handling the empty list. In other words, we only need a `Monoid[Int]`. And that’s easy to come by.

In fact, given a constant functor like we have here, we can turn any `Monoid` into an `Applicative`.